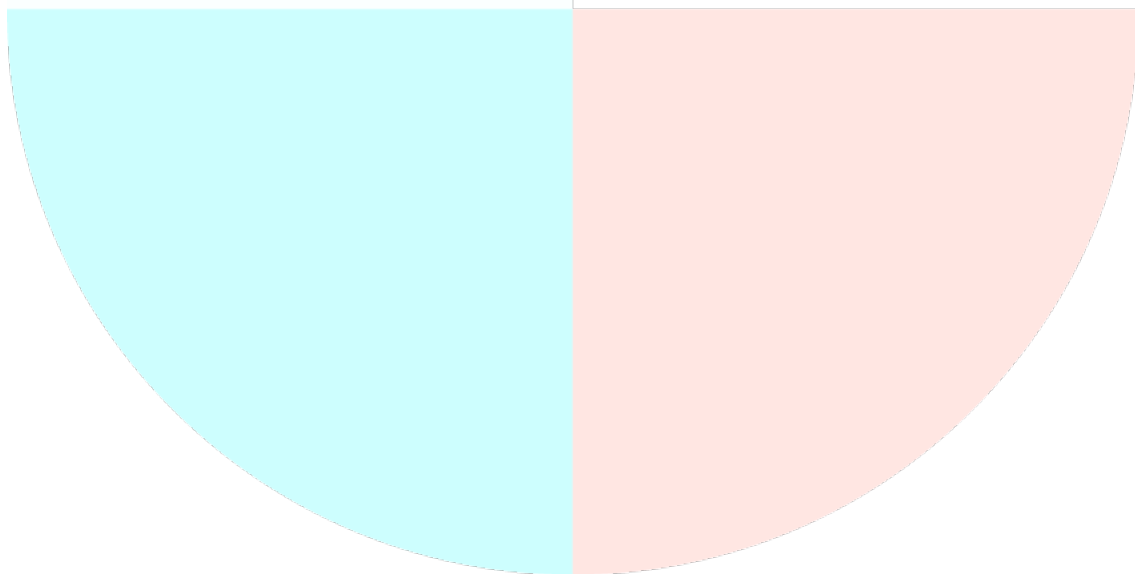


Bloque 3 - Tema 4

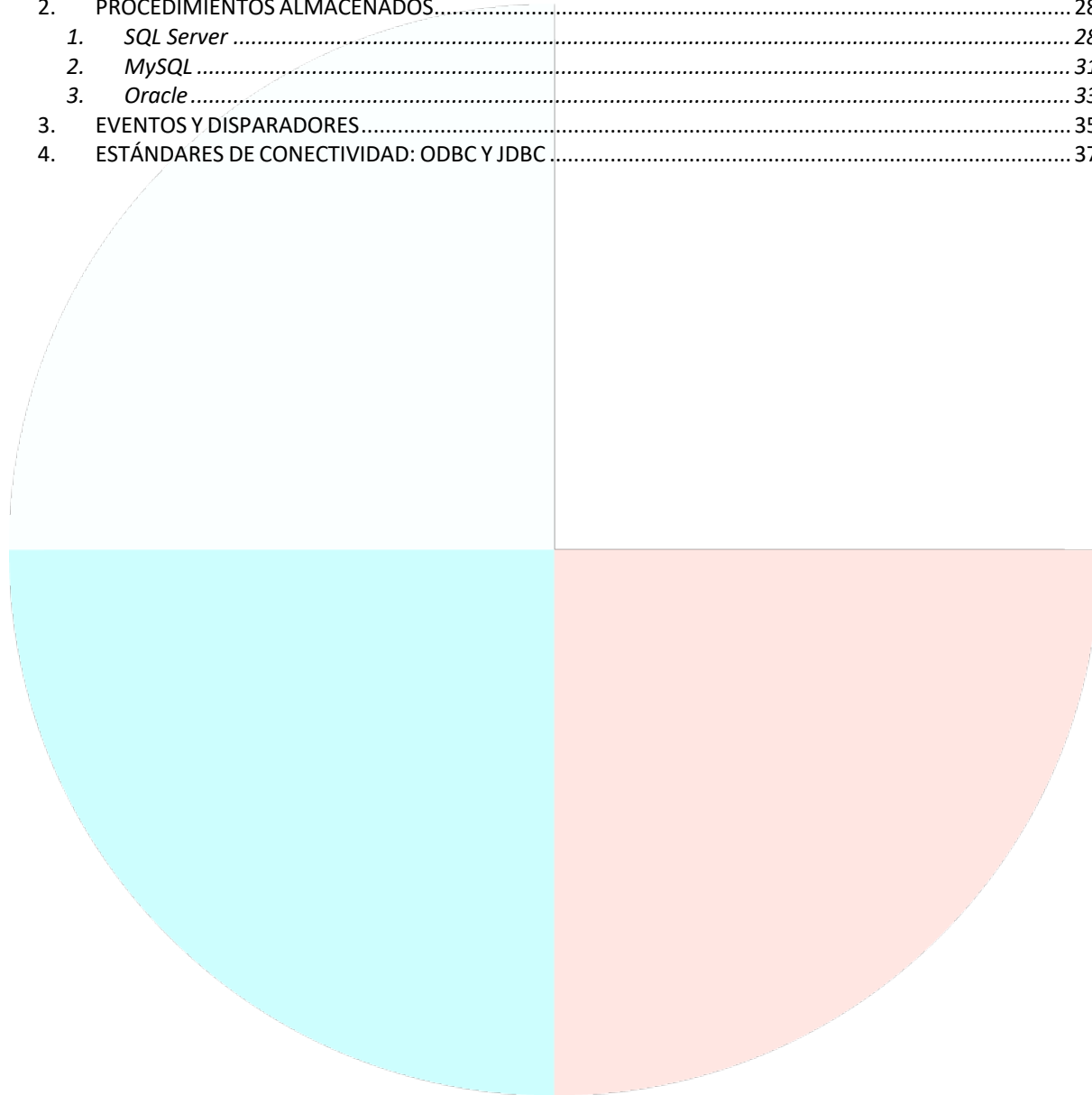
LENGUAJES DE INTERROGACIÓN DE BASES DE DATOS. ESTÁNDAR ANSI SQL. PROCEDIMIENTOS ALMACENADOS. EVENTOS Y DISPARADORES



PREPARACIÓN OPOSICIONES
TÉCNICOS AUXILIARES DE INFORMÁTICA

ÍNDICE

ÍNDICE	2
1. ESTÁNDAR ANSI SQL.....	3
1. Lenguaje de definición de datos (DDL).....	4
2. Lenguaje de manipulación de datos (DML).....	11
3. Lenguaje de control de datos (DCL).....	25
2. PROCEDIMIENTOS ALMACENADOS.....	28
1. SQL Server	28
2. MySQL	31
3. Oracle	33
3. EVENTOS Y DISPARADORES.....	35
4. ESTÁNDARES DE CONECTIVIDAD: ODBC Y JDBC.....	37



1. ESTÁNDAR ANSI SQL

SQL es el lenguaje estándar ANSI/ISO de definición, manipulación y control de bases de datos relaciones. Es un lenguaje declarativo, ya que se indica qué datos obtener, en contraposición con los lenguajes procedimentales, en los que hay que especificar cómo se obtienen los datos.

SQL (STRUCTURES QUERY LANGUAGE, lenguaje estructurado de consultas) es un lenguaje de consulta utilizado para manipular bases de datos relacionales. De los distintos lenguajes que presentaban los primeros SGBDR (Sistemas de Gestión de Bases de Datos Relacionales), ha acabado imponiéndose como estándar, tanto de iure como de facto, el SQL, lenguaje que actualmente ofrecen, aunque con variaciones, la mayor parte de los productos comerciales y que viene siendo objeto, desde hace varios años, de un intenso proceso de normalización.

Fue desarrollado a mediados de los años 70 por IBM. Por esos años se llamaba SEQUEL (Structured English QUery Language) y servía como interfaz para un sistema experimental de base de datos llamado SYSTEM R. Fue introducido por Oracle por primera vez en una base de datos comercial en el año 1979. El lenguaje se utiliza para definir, consultar y actualizar la base de datos, y es el más popular de su estilo.

En 1982, el ANSI (American National Standards Institute) escogió SQL como lenguaje estándar en 1986 y la ISO (Internacional Standards Organization) en 1987. En 1989, el estándar fue objeto de una revisión y una ampliación que dieron lugar al lenguaje que se conoce con el nombre de SQL1 o SQL:1989.

En 1992, el estándar volvió a ser revisado y ampliado considerablemente para cubrir carencias de la versión anterior. Esta nueva versión de SQL se conoce con el nombre de SQL2 o SQL:1992.

En 1999, apareció la esperada versión SQL3 o SQL:1999, en la que, entre otras cosas, se definía formalmente uno de los componentes lógicos de control más útiles: los disparadores.

AÑO	NOMBRE	ALIAS	COMENTARIOS
1986	SQL-86	SQL-87	Primera publicación por ANSI, confirmada por ISO
1989	SQL-89	SQL1	Revisión menor
1992	SQL-92	SQL2	Revisión mayor
1999	SQL:1999	SQL3	Expresiones regulares, consultas recursivas, disparadores y orientación a objetos
2003	SQL:2003		Características XML, objetos sequence y columnas autonuméricas
2005	SQL:2005		Integración con XML
2008	SQL:2008		Disparadores INSTEAD OF

2011	SQL:2011		Datos temporales
2016	SQL:2016		Compatibilidad ficheros JSON

La última publicación de estándar es **ISO/IEC 9075:2016** (IEC significa International Electrotechnical Commission).

Esquema sentencias básicas SQL

DDL	Creación →	CREATE
	Modificación →	ALTER
	Eliminación →	DROP
	Eliminación datos (no tabla) y reseteo contadores →	TRUNCATE
DML	Selección →	SELECT
	Inserción →	INSERT
	Modificación →	UPDATE
	Eliminación →	DELETE
DCL	Validación transacciones →	COMMIT, ROLLBACK, SAVEPOINT
	Autorización →	GRANT, REVOKE

1. Lenguaje de definición de datos (DDL)

Permite la especificación de un conjunto de relaciones y además de información sobre cada una de las relaciones, incluyendo:

- El esquema de cada relación.
- El dominio de los datos asociados a cada atributo.
- Restricciones de integridad.
- El conjunto de índices que se debe mantener para cada relación.
- Información de seguridad y autorización para cada relación.
- La estructura de almacenamiento físico de cada relación en disco.

Para las sentencias de definición utilizaremos una extensión de la Forma Normal de Backus (BNF) para especificar las cláusulas del lenguaje donde:

- < > representa los símbolos no terminales del lenguaje

- ::= es el operador de definición
- [] indica elementos opcionales
- {} agrupa elementos en una fórmula
- | indica una alternativa
- ... indica repetición

De los distintos elementos que puede contener un esquema relacional en SQL, estudiaremos los dominios, aserciones y tablas y se presentarán las opciones y posibilidades que resultan más interesantes.

Tipos de datos

TIPO	DESCRIPCIÓN
CHARACTER (longitud) CHAR(longitud)	Cadena de caracteres de longitud fija
CHARACTER VARYING (longitud) VARCHAR(longitud)	Cadena de caracteres de longitud variable
BIT (longitud)	Cadena de bits de longitud fija
BIT VARYING (longitud)	Cadena de bits de longitud variable
NUMERIC INT	Número decimal
DECIMAL	Número decimal
INTEGER	Número entero
SMALLINT	Número entero pequeño
REAL	Número con coma flotante con precisión predefinida
FLOAT (precisión)	Número con coma flotante con la precisión especificada
DOUBLE PRECISION	Número con coma flotante con más precisión predefinida que la de tipo REAL
DATE	Fecha. Se compone de YEAR, MONTH, DAY
TIME	Horas. Se compone de HOUR, MINUT, SECOND
TIMESTAMP	Fechas y horas. Se compone de YEAR, MONTH, DAY, HOUR, MINUT, SECOND

1. Creación de un esquema

El estándar SQL no dispone de sentencias para la creación de una base de datos. Partiendo de que lo que se pretende es disponer de un conjunto de tablas relacionadas entre sí, se propone una sentencia de creación de esquemas denominada CREATE SCHEMA.

Así pues, un esquema agrupa un conjunto de elementos de la base de datos que son propiedad de un usuario. La sintaxis es:

```
CREATE SCHEMA {<nombre_esquema> | AUTHORIZATION <usuario>}  
[<lista_elementos_esquema>];
```

Esta sentencia permite que un conjunto de tablas <lista_elementos_esquema> se agrupen bajo un mismo nombre, <nombre_esquema>, y que tengan un propietario, <usuario>.

2. Eliminación de un esquema

La sentencia para borrar un esquema es:

```
DROP SCHEMA <nombre_esquema> {RESTRICT | CASCADE};
```

La opción RESTRICT borra el esquema solo si no contiene elemento alguno. Y la opción CASCADE, borra el esquema independientemente de si está vacío o no.

3. Creación de una tabla

Para crear una tabla hay que utilizar la sentencia CREATE TABLE con el siguiente formato:

```
CREATE TABLE nombre_tabla (<definicion_columna>  
    [, <definicion_columna> ...]  
    [, <restricciones_tabla>]  
);
```

Donde definicion_columna es:

<nombre_columna> {<tipo_datos> | <dominio>} [<def_defecto>] [<restricciones_columna>]

El proceso que hay que seguir para crear una tabla es:

- Decidir el nombre de la tabla.
- Dar nombre a cada uno de los atributos que formarán las columnas.
- Asignar a cada una de las columnas un tipo de datos predefinido o bien un dominio definido por el usuario, así como el valor por defecto y, si existen, las restricciones de columna.
- Por último, se especifican las restricciones de la tabla.

Definiciones por defecto

La opción **<def_defecto>** permite especificar qué valor dar a una columna cuando no se especifica dato alguno.

Así DEFAULT tiene el siguiente formato:

DEFAULT {<literal>|<función>|NULL}

Ejemplo: columna sueldo con valor por defecto cero.

sueldo INTEGER DEFAULT 0

Ejemplo: columna sueldo con valor por defecto nulo.

sueldo INTEGER DEFAULT NULL

Restricciones de columna

A cada una de las columnas de una tabla, una vez asignado nombre y definido el dominio en el que toma valores, se pueden imponer ciertas restricciones a satisfacer mediante la opción **<restricción_columna>**:

- **NOT NULL:** la columna no puede tener valores nulos.
- **UNIQUE:** la columna no puede tener valores repetidos (clave candidata).
- **PRIMARY KEY:** la columna no puede tomar valores repetidos ni nulos. Es la clave primaria.
- **REFERENCES** <nombre_tabla> [(<nombre_columna>)]: la columna es clave foránea de la columna de la tabla especificada.
- **CONSTRAINT** [<nombre_restriccion>] CHECK (<condiciones>): la columna tiene que cumplir las condiciones especificadas.

Restricciones de tabla

Además de las restricciones que se definen para las columnas de una tabla, también es posible aplicar restricciones a nivel de tabla mediante **<restricciones_tabla>**, las cuales siempre se tendrán que cumplir. Dichas restricciones pueden ser:

- **UNIQUE** (<nombre_columna> [, <nombre_columna>...]): el conjunto de las columnas especificadas no puede tener valores repetidos (clave candidata).
- **PRIMARY KEY** (<nombre_columna> [, <nombre_columna>...]): el conjunto de las columnas especificadas no puede tener valores nulos ni repetidos. Es la clave primaria.
- **FOREIGN KEY** (<nombre_columna> [, <nombre_columna>...])
REFERENCES <nombre_tabla> [(<nombre_columna> [, <nombre_columna>...])]: el conjunto de las columnas especificadas es una clave ajena que referencia a la clave primaria formada por el conjunto de las columnas de la tabla.
- **CONSTRAINT** [<nombre_restriccion>] CHECK (<condiciones>): la tabla tiene que cumplir las condiciones especificadas.

Tratamiento de claves ajenas cuando las claves primarias referenciadas cambian o se eliminan

SQL nos ofrece la posibilidad de especificar cuando definimos una clave foránea, la política de actuación al modificar o eliminarse la clave primaria a la que hace referencia.

Acabamos de ver la definición de claves foráneas. Ahora ampliamos su definición:

```
FOREIGN KEY <clave_foranea> REFERENCES [<clave_primaria>]
[ON DELETE {NO ACTION | CASCADE | SET DEFAULT | SET NULL}]
[ON UPDATE {NO ACTION | CASCADE | SET DEFAULT | SET NULL}]
```

Donde:

- **NO ACTION:** no se ejecutará ninguna acción.
- **CASCADE:** se actualizará en cascada la clave ajena.
- **SET DEFAULT:** se actualizará la clave ajena al valor por defecto.
- **SET NULL:** se actualizará la clave ajena al valor nulo.

Aserciones

Vistas las restricciones que se pueden especificar al definir una tabla, existen otro tipo de restricciones a nivel global, que se incluirán en este epígrafe. Son las llamadas aserciones, que hacen referencia a una o varias columnas de una o más tablas.

Para definir una aserción utilizamos la sentencia CREATE ASSERTION, con el siguiente formato:

```
CREATE ASSERTION <nombre_asercion> CHECK (<condiciones>;
```

Para borrar una aserción utilizamos la sentencia DROP ASSERTION, con el formato:

```
DROP ASSERTION <nombre_asercion>;
```

4. Modificación de una tabla

Para modificar una tabla utilizaremos la sentencia ALTER TABLE. El formato es el que sigue:

```
ALTER TABLE <nombre_tabla> {<acción_modificar_columna> |
<acción_modificar_restriccion_tabla>;
```

Modificación de una columna

<acción_modificar_columna> se define como:

```
{ADD [COLUMN] <nombre_columna> <def_columna> |
ALTER [COLUMN] <nombre_columna> {SET <def_defecto> | DROP DEFAULT} |
DROP [COLUMN] <nombre_columna> {RESTRICT | CASCADE} }
```


Modificación de una restricción a nivel de tabla

<acción_modificar_restriccion_tabla> tiene el formato que sigue:

```
{ADD <def_restriccion> |  
DROP CONSTRAINT <nombre_restriccion> {RESTRICT|CASCADE} }
```

5. Eliminación de una tabla

Para borrar una tabla, hay que utilizar la sentencia DROP TABLE:

```
DROP TABLE <nombre_tabla> {RESTRICT|CASCADE};
```

Con la opción RESTRICT la tabla no se borrará si está referenciada desde otra tabla o vista.
Con la opción CASCADE todo lo que referencie la tabla se borrará.

6. Creación de una vista

Según la arquitectura ANSI/X3/SPARC y los niveles que define, las tablas forman parte del esquema conceptual.

Ahora en este apartado, utilizamos el concepto de vista para definir los esquemas externos o lógicos.

Para crear una vista es necesario hacer uso de la sentencia CREATE VIEW:

```
CREATE VIEW <nombre_vista> [lista_columnas] AS (consulta)  
[WITH CHECK OPTION];
```

La cláusula WITH CHECK OPTION permite controlar que sólo se admiten operaciones de inserción y modificación que no atenten contra la expresión de consulta que define la vista.

Otro aspecto a tener en cuenta es que, en la actualidad, sólo son actualizables las vistas que proceden de una única tabla (en cuya definición no aparezcan las cláusulas GROUP BY o HAVING).

Las vistas no existen realmente como un conjunto de valores almacenados en la base de datos, son tablas ficticias o derivadas o también denominadas no materializadas, es decir son tablas virtuales. Las vistas se construyen a partir de tablas reales o materializadas que se almacenan en base de datos.

Las ventajas que ofrecen las vistas son:

- **Seguridad:** el usuario final solo percibe una parte de la información.
- **Utilidad:** simplifica la construcción de consultas complejas, es decir, mayor legibilidad.

7. Eliminación de una vista

Para eliminar una vista es necesario hacer uso de la sentencia DROP VIEW:

```
DROP VIEW <nombre_vista> {RESTRICT|CASCADE};
```

Con la opción RESTRICT la vista no se borrará si está referenciada desde otra tabla o vista.

Con la opción CASCADE todo lo que referencie la vista se borrará.

8. Creación de un dominio

Para crear un dominio utilizaremos la sentencia CREATE DOMAIN:

```
CREATE DOMAIN <nombre_dominio> [AS] <tipo_datos>  
[<def_defecto>] [<restricciones_dominio>];
```

Donde restricciones_dominio tiene el siguiente formato:

```
CONSTRAINT [<nombre_restriccion>] CHECK (<condiciones>)
```

Ejemplo: Creación de un dominio para ciudades siendo solo válidas las capitales de Andalucía.

```
CREATE DOMAIN dom_ciudades AS CHAR(50)  
CONSTRAINT ciudades_andaluzas  
CHECK (VALUE IN ("Almeria", "Cádiz", "Córdoba", "Granada", "Huelva", "Jaén", "Málaga",  
"Sevilla"));
```

9. Modificación de un dominio

Para modificar un dominio se utiliza la sentencia ALTER DOMAIN con el siguiente formato:

```
ALTER DOMAIN <nombre_dominio> {<acción_modificar_dominio>|  
<acción_modificar_restriccion_dominio>}
```

Donde:

- <acción_modificar_dominio> := SET { <def_defecto>|DROP DEFAULT}
- <acción_modificar_restriccion_dominio> := {ADD <restricción_dominio>|
DROP CONSTRAINT <nombre_restriccion>}

Ejemplo: Modificar dominio anterior sin contar con la ciudad Almeria.

```
ALTER DOMAIN dom_ciudades DROP CONSTRAINT ciudades_andaluzas;
```

```
ALTER DOMAIN dom_ciudades ADD CONSTRAINT ciudades_andaluzas  
CHECK (VALUE IN("Cádiz", "Córdoba", "Granada", "Huelva", "Jaén", "Málaga", "Sevilla"));
```

10. Eliminación de un dominio

Para borrar un dominio definido por el usuario, se utiliza la sentencia DROP DOMAIN:

```
DROP DOMAIN <nombre_dominio> {RESTRICT|CASCADE}
```

Teniendo en cuenta que:

- **RESTRICT** permite el borrado del dominio solo si no se utiliza en ningún sitio.
- **CASCADE** borra el dominio aunque esté referenciado y pone el tipo de datos del dominio en sustitución del mismo.

11. Creación de un índice

Para crear un índice en base de datos utilizamos la sentencia CREATE INDEX:

```
CREATE INDEX <nombre_indice>  
ON <tabla> (<columna1> [, <columna2> ...])
```

Asimismo, podemos crear un índice ÚNICO para una tabla, el cual no permitirá valores duplicados:

```
CREATE UNIQUE INDEX <nombre_indice>  
ON <tabla> (<columna1> [, <columna2> ...])
```

12. Eliminación de un índice

Para borrar un índice en base de datos utilizamos la sentencia DROP INDEX:

```
DROP INDEX <nombre_indice>
```

13. Eliminación de los datos de una tabla

Para borrar el contenido de una tabla utilizamos la sentencia TRUNCATE TABLE:

```
TRUNCATE TABLE <nombre_tabla>
```

2. Lenguaje de manipulación de datos (DML)

Una vez creada la base de datos con sus tablas, habrá que poder insertar, modificar y borrar los valores de las filas de las tablas. Para ello SQL nos ofrecen las siguientes sentencias:

- **INSERT**: insertar datos.
- **UPDATE**: actualizar datos.
- **DELETE**: borrar datos.

Y además de eso, debemos poder hacer consultas a la base de datos. La sentencia SELECT nos permitirá recuperar datos de la base de datos.

1. Inserción de datos: INSERT

Para almacenar datos en una tabla de la base de datos hacemos uso de la sentencia INSERT INTO VALUES, con el siguiente formato:

```
INSERT INTO <nombre_tabla> [(<columnas>)]  
{VALUES  
  (<v1>|DEFAULT|NULL, .., {<vn>|DEFAULT|NULL}) | <consulta>;
```

Los valores v_1 a v_n se tienen que corresponder con las columnas definidas en la creación de la tabla y en el mismo orden. Si para un valor queremos introducir el valor por defecto, lo indicaremos mediante DEFAULT y si deseamos que tome el valor nulo, a través de NULL.

También se pueden insertar valores como resultado de una consulta SELECT.

2. Eliminación de datos: DELETE

Si queremos borrar datos (filas) de una tabla utilizamos la sentencia DELETE FROM WHERE y definimos las condiciones a satisfacer por las filas a borrar:

```
DELETE FROM <nombre_tabla>  
[WHERE <condiciones>;
```

Si lo que se pretende es eliminar todos los datos de la tabla basta con no especificar la cláusula WHERE.

Ejemplo: Eliminar los empleados nacidos antes de 1955.

```
DELETE FROM empleados WHERE fecha_nacimiento < '01/01/1955';
```

Ejemplo: Eliminar la tabla empleados.

```
DELETE FROM empleados;
```

3. Modificación de datos: UPDATE

Si lo que pretendemos es modificar los valores de algunas filas de una tabla haremos uso de la sentencia UPDATE SET WHERE, con el siguiente formato:

```
UPDATE <nombre_tabla>  
SET <nombre_columna> = {<expresión>|DEFAULT|NULL}  
  [, <nombre_columna> = {<expresión>|DEFAULT|NULL} ...]  
[WHERE <condiciones>;
```

Ejemplo: Actualizar el campo variable de la tabla empleados a 1000 si su antigüedad supera los 3 años.

```
UPDATE Empleados  
SET variable=1000  
WHERE tiempo>=3;
```

Ejemplo: Actualizar el sueldo de todos los empleados un 1,25%.

```
UPDATE Empleados  
SET sueldo=sueldo*1,0125
```

4. Recuperación de datos: SELECT

Para llevar a cabo consultas sobre una base de datos relacional, el lenguaje SQL propone la sentencia SELECT, que consta de las siguientes cláusulas:

```
SELECT <nombre_columna_seleccionada> [ [AS] <col_renombrada>]  
[, nombre_columna_seleccionada> [ [AS] <col_renombrada>]...]  
FROM <tabla> [ [AS] <tabla_renombrada>]  
[WHERE <condiciones>]
```

La opción **AS** permite renombrar una columna (operador renombramiento), es decir, actúa como un alias.

Por otro lado, el asterisco (*) después de la cláusula SELECT indica que queremos visualizar todas las columnas de la tabla.

Ejemplo: Mostrar todos los clientes.

```
SELECT * FROM Clientes;
```

Ejemplo: Mostrar el nombre, apellidos y dirección de los clientes.

```
SELECT nombre, apellidos, dirección FROM Clientes;
```

Ahora bien, si con SELECT seleccionamos las columnas de una tabla, con WHERE se determinan las filas a seleccionar, es decir, las filas que cumplen la condición especificada.

Para definir las condiciones de la cláusula WHERE, usamos los operadores.

Operadores

Tanto en la cláusula WHERE (condiciones de fila) como en la cláusula HAVING (condiciones de grupo) se pueden utilizar una serie de operadores:

- **De comparación:**
 - <: menor que.
 - <=: menor o igual que.

- >: mayor que.
- >=: mayor o igual que.
- =: igual que.
- !=: distinto.
- **Lógicos:**
 - AND: conjunción de condiciones.
 - OR: disyunción de condiciones.
 - NOT: negación de condiciones.

El orden de precedencia o evaluación de los operadores es el siguiente salvo expresiones con paréntesis:

- 1º: todos los operadores de comparación.
- 2º: NOT.
- 3º: AND.
- 4º: OR.

Por tanto, se aconseja el uso de paréntesis para forzar la prioridad deseada de los operadores.

Predicados

Se detallan a continuación predicados a utilizar en la cláusula WHERE:

- **BETWEEN valor AND valor**: intervalos de valores. Establece una comparación dentro un intervalo cerrado. Se puede utilizar **NOT BETWEEN valor AND valor** para establecer una comparación fuera del intervalo.

```
SELECT <nombre_columnas>
FROM <tabla>
WHERE <columna> BETWEEN <limite> AND <limite2>;
```

Ejemplo: Obtener los expedientes con entrada en 2018.

```
SELECT * FROM expedientes
WHERE fecha_entrada BETWEEN '01/01/2018' AND '31/12/2018';
```

- **LIKE**: búsqueda por patrones. Establece una comparación entre cadenas de caracteres (**NOT LIKE** como contraposición), empleando los siguientes comodines:
 - %: sustituye a una cadena de caracteres de cualquier longitud, de cero a varios caracteres.
 - _: sustituye a un único carácter.

```
SELECT <nombre_columnas>
FROM <tabla>
WHERE <columna> LIKE <característica>;
```

Ejemplo: Obtener los ciudadanos cuyo primer apellido comienza por G.

```
SELECT *  
FROM ciudadanos  
WHERE apellido1 LIKE "G%";
```

- **IN:** comprueba la pertenencia de un valor a un conjunto dado. **NOT IN** como contraposición.

```
SELECT <nombre_columnas>  
FROM <tabla>  
WHERE <columna> [NOT] IN (<valor1>, ..., <valorN>);
```

Ejemplo: Obtener las licencias que están denegadas o archivadas.

```
SELECT * FROM licencias  
WHERE estado IN ("DENEGADO", "ARCHIVADO");
```

- **IS NULL:** establece la comparación de un atributo con el valor nulo. **IS NOT NULL** para la comparación contraria.

```
SELECT <nombre_columnas>  
FROM <tabla>  
WHERE <columna> IS [NOT] NULL;
```

Ejemplo: Obtener los números de expediente que se han resuelto.

```
SELECT num_expediente FROM expedientes  
WHERE fecha_resolucion IS NOT NULL;
```

Operador CASE

Es el operador estándar de elección entre múltiples valores. Admite dos sintaxis distintas:

- Devuelve el valor ret1 si expresión se evalúa a val1 y a continuación termina. Si no, devuelve el valor ret2 si expresión se evalúa a val2 y así sucesivamente. Si ninguna de las igualdades anteriores se satisface, se devuelve retn. Si no existe la cláusula else y ninguna de las igualdades anteriores se ha cumplido, entonces devuelve null.

```
CASE expresion WHEN val1 THEN ret1  
                WHEN val2 THEN ret2  
                ...  
                ELSE retn  
END
```

- Devuelve el valor ret1 si expresion1 se evalúa a cierto y a continuación termina. Si no, devuelve el valor ret2 si expresión se evalúa a cierto y así sucesivamente. Si no existe la cláusula else y ninguna de las expresiones anteriores se ha evaluado a cierto devuelve el valor null.

```
CASE WHEN expresion1 THEN ret1  
      WHEN expresion2 THEN ret2
```

```
...
ELSE retn
END
```

Ejemplo: Mostrar el nombre descriptivo “Ingreso Libre” si el campo idFormaAcceso toma el valor “L” o “Promoción Interna” si toma el valor “P”.

```
SELECT IdProceso, CASE IdFormaAcceso WHEN 'L' THEN 'Ingreso Libre'
                                WHEN 'P' THEN 'Promoción interna'
                                END AS FormaAcceso
FROM Proceso;
```

DISTINCT

Para que en una consulta no aparezcan filas repetidas, usamos la palabra clave DISTINCT inmediatamente después de SELECT.

ALL

Si optamos porque aparezcan filas repetidas (opción por defecto) la palabra clave ALL será la usada. Si tras SELECT no especificamos nada será ALL la palabra clave por defecto.

```
SELECT [ALL] DISTINCT <columnas>
FROM <tabla>
[WHERE <condiciones>]
```

Funciones de agregación

SQL ofrece una serie de funciones de agregación para realizar diferentes operaciones con los datos:

- **COUNT:** número total de filas seleccionadas.
- **SUM:** suma los valores de una columna.
- **MIN:** valor mínimo de una columna.
- **MAX:** valor máximo de una columna.
- **AVG:** valor medio de una columna.

Las funciones de agregación se aplican a una columna concreta, excepto la función de agregación COUNT que se aplica todas las columnas.

- **COUNT(*):** número de filas que cumplan las condiciones.
- **COUNT(DISTINCT <nombre_columna>):** número de filas no nulo ni repetidos.
- **COUNT(<nombre_columna>):** número de filas con valor no nulo.

Ejemplo: obtener el número de clientes de Granada.

```
SELECT COUNT(*) AS num_clientes FROM clientes WHERE ciudad="Granada";
```


Subconsultas

Una subconsulta es una consulta incluida dentro de una cláusula WHERE (o HAVING) de otra consulta.

Ejemplo: Obtener los clientes que más han comprado.

```
SELECT * FROM clientes
WHERE comprado = (SELECT MAX(comprado) FROM clientes).
```

Predicados aplicables a subconsultas:

- **Cuantificadores:** **ANY** (alguno), **ALL** (todos). Se utiliza para comprobar si alguno o todos los valores de la consulta satisface o satisfacen la comparación con la consulta interna definida.

```
SELECT <nombre_columnas>
FROM <tabla>
WHERE <columna> <operador_comparacion> {ALL|ANY} <subconsulta>;
```

Ejemplo: Obtener los nombres de los proyectos en la que los sueldos de todos los empleados asignados no superen el precio del proyecto.

```
SELECT nombre_proyecto FROM proyectos
WHERE precio > ALL ( SELECT sueldo FROM empleado
                     WHERE proyectos.idproyecto = empleado.idproyecto);
```

Ejemplo: Obtener los nombres de los proyectos en la que algún empleado tiene un sueldo superior al precio del proyecto al que está asignado.

```
SELECT nombre_proyecto FROM proyectos
WHERE precio < ANY ( SELECT sueldo FROM empleado
                    WHERE proyectos.idproyecto = empleado.idproyecto);
```

- **Existencial:** **EXISTS**. Indica la existencia o no de un conjunto. Se permite utilizar **NOT EXISTS**. Se utiliza para comprobar si la consulta interna devuelve algún resultado o no.

```
SELECT <nombre_columnas>
FROM <tabla>
WHERE [NOT] EXISTS <subconsulta>;
```

Ejemplo: Obtener los funcionarios asignados a algún proyecto.

```
SELECT * FROM funcionarios
WHERE EXISTS (SELECT * FROM proyectos
              WHERE proyectos.idproyecto=funcionarios.idproyecto);
```

Ordenación de datos ORDER BY

Hasta ahora los datos obtenidos en la cláusula SELECT son mostrados en un orden preestablecido. Sin embargo, si queremos definir un orden concreto hay que utilizar la cláusula ORDER BY, que especifica la columna (o columnas) sobre la cual se deben ordenar las

filas obtenidas por la consulta. El sentido de la ordenación puede ser ascendente (ASC) o descendente DESC, estableciéndose como sentido por defecto el ascendente.

```
SELECT <nombre_columnas>
FROM <tabla>
[WHERE <condiciones>]
ORDER BY <nombre_columna_orden [DESC]
[, <nombre_columna_orden [DESC] ...];
```

Hay que destacar que los campos que se especifiquen en la cláusula ORDER BY tienen que haber sido previamente definidos en la cláusula SELECT. Por defecto, el orden es ascendente.

Ejemplo: Obtener las autorizaciones ordenadas por fecha de entrada.

```
SELECT fecha_entrada, num_autorizacion FROM autorizaciones
ORDER BY fecha_entrada;
```

Agrupación de filas GROUP BY y HAVING

El estándar SQL pone a disposición dos cláusulas que permiten organizar las filas en grupos:

- **GROUP BY:** permite agrupar filas de una tabla según la columna especificada en la cláusula.
- **HAVING:** especifica condiciones de búsqueda para grupos de filas. Su funcionalidad es similar a la cláusula WHERE pero en lugar de aplicarse a filas se aplica a cada uno de los grupos obtenidos con la cláusula anterior.

```
SELECT <nombre_columnas>
FROM <tabla>
[WHERE <condiciones>]
GROUP BY <columnas_sobre_las_que_agrupar>
[HAVING <condición_para_cada_grupo>];
```

En la cláusula SELECT solo pueden figurar los campos que se especifiquen en la cláusula GROUP BY y funciones de agregación.

Ejemplo: Obtener el sueldo medio de los funcionarios por CCAA y provincia.

```
SELECT nombre_CCAA, nombre_provincia, AVG(sueldo)
FROM funcionarios
GROUP BY nombre_CCAA, nombre_provincia;
```

Ejemplo: Obtener el sueldo medio de los funcionarios por CCAA y provincia cuyo sueldo medio sea superior a 25.000.

```
SELECT nombre_CCAA, nombre_provincia, AVG(sueldo)
FROM funcionarios
GROUP BY nombre_CCAA, nombre_provincia HAVING AVG(sueldo) > 25000;
```

Ahora conviene recapitular lo visto sobre la recuperación de datos:

```
SELECT [ALL|DISTINCT] <expresión|*>
FROM <tablas|vistas>
[WHERE <condición>]
[GROUP BY <lista_de_atributos>]
[HAVING <condición_de_grupo>]
ORDER BY <nombre_columna_orden [DESC]
        [, <nombre_columna_orden [DESC] ...];
```

CLÁUSULA	SIGNIFICADO
SELECT	Especifica la información que se desea obtener
DISTINCT	Elimina valores repetidos
FROM	Indica las tablas o vistas en las que se encuentran los atributos implicados en la consulta
WHERE	Especifica la condición de búsqueda
GROUP BY	Permite agrupar los resultados
HAVING	Especifica una condición de grupo
ORDER BY	Permite ordenar el resultado de la consulta

En relación con el álgebra relacional visto en el tema 2 de este bloque SELECT se corresponde con el operador proyección, FROM con el operador producto cartesiano y WHERE con el operador selección:

```
SELECT A1, A2, ..., Ak
FROM R1, R2, ..., Rn
WHERE p;
```

$$\Pi_{A1, A2, \dots, Ak} (\sigma_p (R1 \times R2 \times \dots \times Rn))$$

$$\Pi_{A1, A2, \dots, Ak} (R) \longrightarrow \text{SELECT } A1, A2, \dots, Ak \text{ FROM } R;$$

$$\sigma_p (R) \longrightarrow \text{SELECT } * \text{ FROM } R \text{ WHERE } p;$$

$$R1 \times R2 \longrightarrow \text{SELECT } * \text{ FROM } R1, R2;$$

Consultas con más de una tabla

Con lo visto hasta ahora solo podemos consultar los datos de una tabla individual y, como mucho, involucrar dos tablas a través de una subconsulta.

TOY			CAT	
toy_id	toy_name	cat_id	cat_id	cat_name
1	pelota	3	1	Kitty
2	muelle	NULL	2	Hugo
3	ratón	1	3	Sam
4	ratón	4	4	Misty
5	pelota	1		

Sin embargo, lo habitual es la consulta de más de una tabla y utilizar columnas de dichas tablas. Para ello, disponemos de la cláusula FROM. Vemos ahora las diferentes posibilidades:

- **Producto cartesiano (CROSS JOIN)**: se obtienen todos los registros de la tabla de la izquierda combinados con cada uno de los registros de la tabla de la derecha. No se incluye ninguna condición en la cláusula WHERE que filtre el resultado. El formato se puede expresar de dos formas:

```
SELECT <nombre_columnas>
FROM <tabla1>, <tabla2>;
```

```
SELECT <nombre_columnas>
FROM <tabla1> CROSS JOIN <tabla2>;
```

```
SELECT *
FROM toy
CROSS JOIN cat
```

toy_id	toy_name	cat_id	cat_id	cat_name
1	pelota	3	1	Kitty
2	muelle	NULL	1	Kitty
3	ratón	1	1	Kitty
4	ratón	4	1	Kitty
5	pelota	1	1	Kitty
1	pelota	3	2	Hugo
2	muelle	NULL	2	Hugo
3	ratón	1	2	Hugo
4	ratón	4	2	Hugo
5	pelota	1	2	Hugo
...	...	...	...	...

- **Combinación (JOIN)**: permite crear una sola tabla a partir de las tablas especificadas en la cláusula FROM, teniendo en cuenta que tenemos que hacer que coincidan los valores de las columnas relacionadas en ambas tablas (en otro caso sería un producto cartesiano).

```
SELECT <nombre_columnas>
FROM <tabla1>, <tabla2>
WHERE <condiciones>;
```

Si <tabla1> y <tabla2> son la misma tabla, se le denomina **SELF JOIN**.

Ejemplo: Obtener los ciudadanos que ha solicitado autorizaciones.

```
SELECT nombre, apellidos, ciudadanos.nif, num_autorizacion, fecha_entrada
FROM ciudadanos, autorizaciones WHERE ciudadanos.nif = autorizaciones.nif;
```

Es importante destacar que al trabajar con más de una tabla pueden coincidir los nombres de las columnas. Es por ello, que es obligatorio especificar a qué tabla corresponde cada columna (ciudadano.nif). También, pueden utilizarse un alias en la cláusula FROM.

Ejemplo: Obtener los ciudadanos que ha solicitado autorizaciones.

```
SELECT nombre, apellidos, c.nif, num_autorizacion, fecha_entrada
FROM ciudadanos c, autorizaciones a WHERE c.nif = a.nif;
```

Otra forma de expresar la combinación vista sería:

```
SELECT <nombre_columnas>
FROM <tabla1> JOIN <tabla2>
{ON <condiciones> |
  USING (<nombre_columna> [, <nombre_columna> ...])}
[WHERE <condiciones>];
```

	toy_id	toy_name	cat_id	cat_id	toy_name
SELECT *	5	pelota	1	1	Kitty
FROM toy	3	ratón	1	1	Kitty
JOIN cat	1	pelota	3	3	Sam
ON toy.cat_id = cat.cat_id	4	ratón	4	4	Misty

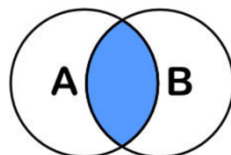
Siguiendo el ejemplo anterior la consulta se expresaría así:

```
SELECT nombre, apellidos, c.nif, num_autorizacion, fecha_entrada
FROM ciudadanos c JOIN autorizaciones a ON c.nif=a.nif;
```

```
SELECT nombre, apellidos, c.nif, num_autorizacion, fecha_entrada
FROM ciudadanos c JOIN autorizaciones a USING (nif);
```

Así, la opción JOIN permite vincular las tablas y mediante la opción ON nos permite expresar las condiciones de igualdad.

- **Combinación natural (NATURAL JOIN):** su funcionamiento es el mismo que en el álgebra relacional, es decir, establece una comparación de igualdad entre las columnas con el mismo nombre (se realiza de forma automática sin necesidad de especificar las columnas a comparar) y luego elimina las columnas repetidas. Es un caso especial de INNER JOIN.



```
SELECT <nombre_columnas>
FROM <tabla1> NATURAL JOIN <tabla2>
[WHERE <condiciones>];
```

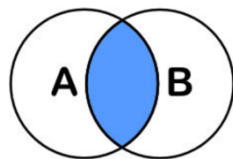
```
SELECT *
FROM toy
NATURAL JOIN cat
```

cat_id	toy_id	toy_name	toy_name
1	5	pelota	Kitty
1	3	ratón	Kitty
3	1	pelota	Sam
4	4	ratón	Misty

Volvamos, pues, al ejemplo visto antes:

```
SELECT nombre, apellidos, num_autorizacion, fecha_entrada
FROM ciudadanos NATURAL JOIN autorizaciones;
```

- **Combinación interna (INNER JOIN)**: solo se queda con las filas con valores idénticos de las tablas que compara, descartando los resultados que no se corresponden en ambas. Hay que especificar las columnas a comparar (no es automático como NATURAL JOIN). Su uso puede suponer la pérdida de información, es decir, si existen valores nulos en alguna de las tablas al hacer la comparación. Formato:



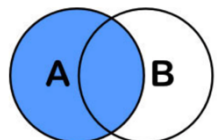
```
SELECT <nombre_columnas>
FROM <tabla1> INNER JOIN <tabla2>
{ON <condiciones> |
  USING (<nombre_columna> [, <nombre_columna> ...])}
[WHERE <condiciones>;
```

- **Combinación externa (OUTER JOIN)**: permite obtener todos los valores de la tabla puesta a derecha o a la izquierda o ambas (evitamos perder filas por valores nulos). Formato general:

```
SELECT <nombre_columnas>
FROM <tabla1> {LEFT | RIGHT | FULL} [OUTER] JOIN <tabla2>
{ON <condiciones> |
  USING (<nombre_columna> [, <nombre_columna> ...])}
[WHERE <condiciones>;
```

LEFT JOIN

```
SELECT <nombre_columnas>
FROM <tabla1> LEFT [OUTER] JOIN <tabla2>
ON tabla1.columna=tabla2.columna;
```

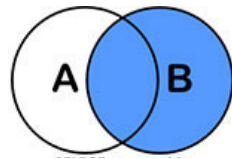


```
SELECT *
FROM toy
LEFT JOIN cat
ON toy.cat_id = cat.cat_id
```

toy_id	toy_name	cat_id	cat_id	toy_name
5	pelota	1	1	Kitty
3	ratón	1	1	Kitty
1	pelota	3	3	Sam
4	ratón	4	4	Misty
2	muelle	NULL	NULL	NULL

RIGHT JOIN

```
SELECT <nombre_columnas>
FROM <tabla1> RIGHT [OUTER] JOIN <tabla2>
ON tabla1.columna=tabla2.columna;
```

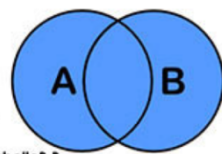


```
SELECT *
FROM toy
RIGHT JOIN cat
ON toy.cat_id = cat.cat_id
```

toy_id	toy_name	cat_id	cat_id	toy_name
5	pelota	1	1	Kitty
3	ratón	1	1	Kitty
NULL	NULL	NULL	2	Hugo
1	pelota	3	3	Sam
4	ratón	4	4	Misty

FULL JOIN

```
SELECT <nombre_columnas>
FROM <tabla1> FULL [OUTER] JOIN <tabla2>
ON tabla1.columna=tabla2.columna;
```



```
SELECT *
FROM toy
FULL JOIN cat
ON toy.cat_id = cat.cat_id
```

toy_id	toy_name	cat_id	cat_id	toy_name
5	pelota	1	1	Kitty
3	ratón	1	1	Kitty
NULL	NULL	NULL	2	Hugo
1	pelota	3	3	Sam
4	ratón	4	4	Misty
2	muelle	NULL	NULL	NULL

También se considera válido indicar FULL JOIN en lugar de FULL OUTER JOIN.

Ejemplo: supongamos que queremos obtener los funcionarios que prestan servicios en las Oficinas. Así tenemos el funcionario "Pepe" que está en excedencia y no está asignado a ninguna Oficina y la Oficina "Motril" que es de nueva creación y aún no tiene funcionarios asignados a ella.

Combinación natural interna

```
SELECT e.codigo, e.nombre, e.apellidos, d.nombre_oficina, d.ciudad
FROM funcionarios e NATURAL JOIN oficinas d;
```

Con este ejemplo, no se obtiene el funcionario Pepe y ni la Oficina de Motril.

Combinación externa izquierda

```
SELECT e.codigo, e.nombre, e.apellidos, d.nombre_oficina, d.ciudad
FROM funcionarios e LEFT JOIN oficinas d;
```

Con este ejemplo, se obtiene el funcionario Pepe pero no la Oficina de Motril.

Combinación externa derecha

```
SELECT e.codigo, e.nombre, e.apellidos, d.nombre_oficina, d.ciudad
FROM funcionarios e RIGHT JOIN oficinas d;
```

Con este ejemplo, no se obtiene el funcionario Pepe pero sí la Oficina de Motril.

Combinación externa plena o completa

SELECT e.codigo, e.nombre, e.apellidos, d.nombre_oficina, d.ciudad

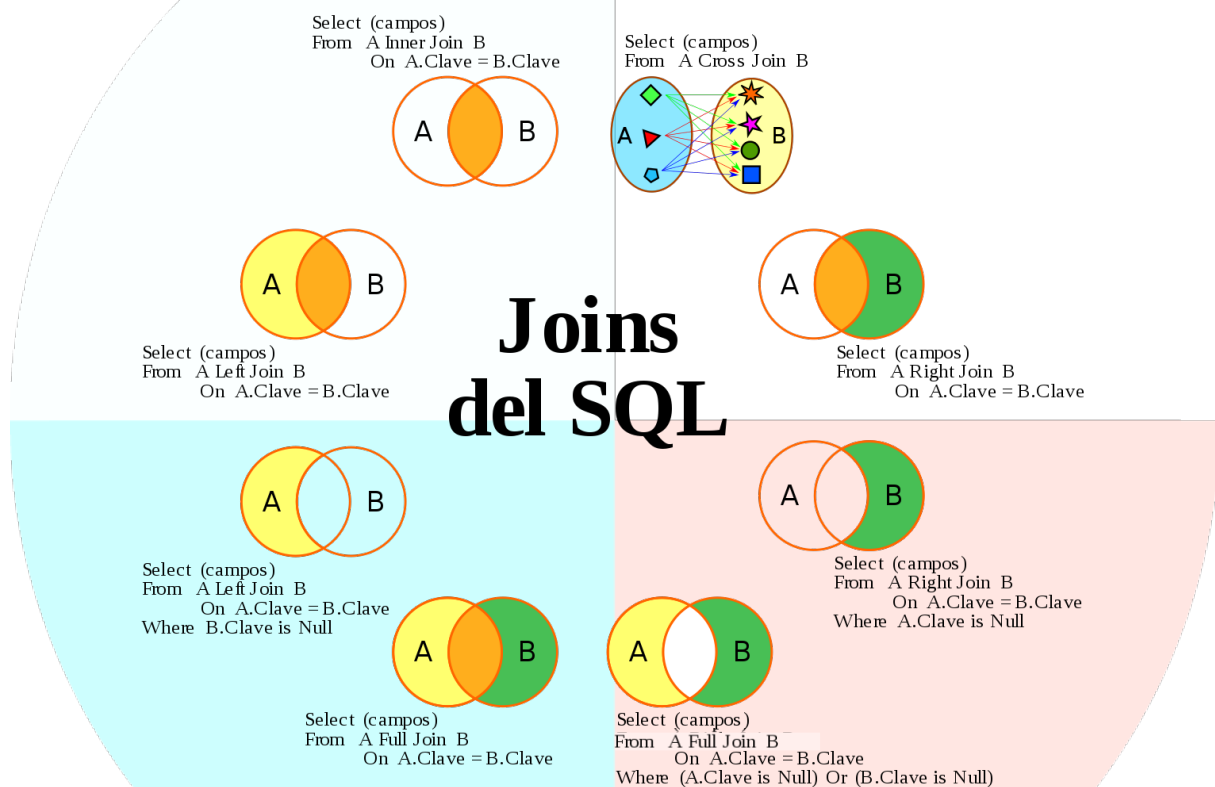
FROM funcionarios e FULL OUTER JOIN oficinas d;

Con este ejemplo, se obtiene el funcionario Pepe y la Oficina de Motril.

Por último, podemos combinar más de dos tablas, con el siguiente formato:

```
SELECT <nombre_columnas>
FROM (<tabla1> JOIN <tabla2> {ON <condiciones>})
JOIN <tabla3> {ON <condiciones>}
```

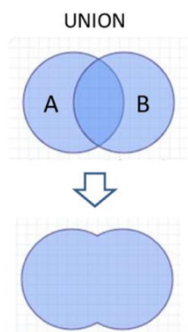
Como resumen de JOINS:



5. Unión de consultas

La cláusula UNION permite unir dos o más consultas. Tiene el siguiente formato:

```
SELECT <nombre_columnas>
FROM <tabla>
[WHERE <condiciones>]
UNION [ALL]
SELECT <nombre_columnas>
FROM <tabla>
[WHERE <condiciones>]
```



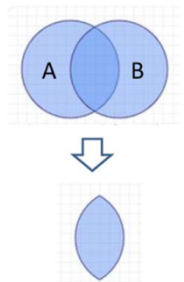
Devuelve las filas que están en una u otra tabla y además las que están en ambas (sólo una vez). Si se indica la opción ALL devolverá las filas repetidas. Hay que tener en cuenta que las columnas deben estar definidas sobre dominios compatibles.

6. Intersección de consultas

La cláusula INTERSECT permite unir dos o más consultas. Tiene el siguiente formato:

```
SELECT <nombre_columnas>  
FROM <tabla>  
[WHERE <condiciones>]  
INTERSECT [ALL]  
SELECT <nombre_columnas>  
FROM <tabla>  
[WHERE <condiciones>]
```

INTERSECTION



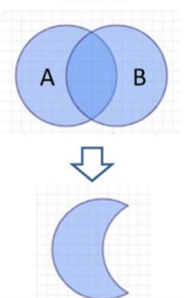
Devuelve las filas que están en ambas tablas a la vez. Si se indica la opción ALL devolverá las filas repetidas. Hay que tener en cuenta que las columnas deben estar definidas sobre dominios compatibles.

7. Diferencia de consultas

La cláusula EXCEPT permite obtener la diferencia de dos o más consultas. Tiene el siguiente formato:

```
SELECT <nombre_columnas>  
FROM <tabla>  
[WHERE <condiciones>]  
EXCEPT [ALL]  
SELECT <nombre_columnas>  
FROM <tabla>  
[WHERE <condiciones>]
```

MINUS



Devuelve las filas que están en la primera tabla y no en la segunda. Si se indica la opción ALL devolverá las filas repetidas. Hay que tener en cuenta que las columnas deben estar definidas sobre dominios compatibles.

3. Lenguaje de control de datos (DCL)

Este lenguaje se encarga fundamentalmente del control de acceso a los datos y de la integridad de estos. De tal forma que se pueden establecer mecanismos de control para resolver problemas de concurrencia de usuarios y garantizar la seguridad de los datos.

Transacciones

Es una unidad lógica de trabajo, es decir, es un conjunto de sentencias que se ejecutan como si fuesen una única sentencia. Además, existe una relación entre las sentencias que forman parte de una transacción, de tal forma que la no ejecución de una sentencia supone que carezca de sentido la ejecución de las demás.

Las transacciones se inician de forma implícita al ejecutar sentencias como CREATE, ALTER, INSERT, UPDATE o de forma explícita con la sentencia SET TRANSACTION.

La finalización de una transacción debe ser explícita con una de las siguientes sentencias:

- **COMMIT**: confirma los cambios realizados en la base de datos durante la ejecución de la transacción.
- **ROLLBACK**: deshace todos los cambios que se hayan producido en la base de datos y se vuelve al estado anterior al inicio de la ejecución de la transacción.
- **SAVEPOINT**: define un punto de recuperación en una transacción para poder revertirla hasta dicho punto mediante ROLLBACK.

Autorizaciones

El acceso a la base de datos conlleva que el usuario disponga de los privilegios necesarios para llevar a cabo las acciones que desea.

Para autorizar se utiliza la siguiente sentencia:

GRANT <privilegios> ON <objeto> TO <usuarios> [WITH GRANT OPTION];

Donde privilegios puede ser:

- ALL PRIVILEGES: otorga todos los privilegios sobre el objeto especificado.
- USAGE: permite la utilización del objeto.
- SELECT: consultas.
- INSERT: inserciones.
- UPDATE: modificaciones.
- DELETE: eliminaciones.

Objeto se corresponde con un:

- DOMAIN: dominio.
- TABLE: tabla.
- VIEW: vista.

Usuarios puede ser una lista de usuarios concretos o todos (PUBLIC).

Por último, la opción **WITH GRANT OPTION** posibilita al usuario que se autoriza poder, a su vez, autorizar a otros usuarios con los mismos privilegios con los que ha sido autorizado.

Revocaciones

En el caso de que queramos retirar los privilegios usaremos la sentencia:

```
REVOKE [GRANT OPTION FOR] <privilegios> ON <objeto> FROM <usuarios>;
```

Donde privilegios y usuarios es igual que para la sentencia GRANT. Y la opción GRANT OPTION FOR se utiliza para retirar el derecho a autorizar a otros usuarios.



2. PROCEDIMIENTOS ALMACENADOS

Un procedimiento almacenado (stored procedure) es un programa que se almacena físicamente en una base de datos. Su implementación concreta varía en función del gestor de bases de datos concreto.

Como principal ventaja, los procedimientos almacenados son ejecutados, en respuesta a la petición de un usuario, directamente en el SGBD. Esto conlleva que el acceso a los datos es directo y que solo devuelve al usuario el resultado, aumentando la eficiencia.

Los procedimientos almacenados se usan típicamente para la validación de datos, para encapsular procesos complejos o que requieren un coste computacional elevado.

Otra ventaja, es que los procedimientos almacenados permiten definir la lógica de negocio de nuestra aplicación o, al menos, parte de ella.

Sintaxis básica:

```
CREATE [OR REPLACE] PROCEDURE nombre(parámetros)
BEGIN
    cuerpo;
END
```

```
CREATE [OR REPLACE] FUNCTION nombre(parámetros)
BEGIN
    cuerpo;
    RETURN valor;
END
```

Cada parámetro tiene la siguiente sintaxis:

parameter_name [IN | OUT | IN OUT] datatype_name

1. SQL Server

Para crear un procedimiento almacenado usamos la sintaxis que se describe a continuación:

```
CREATE [ OR ALTER ] { PROC | PROCEDURE } [schema_name.] procedure_name
[ { @parameter data_type } [= default ] [ OUT | OUTPUT | READONLY ] ] [ ,...n ]
AS { [ BEGIN ] sql_statement [;] [ ...n ] [ END ] }
[;]
```

Ejemplo:

```
CREATE PROCEDURE HumanResources.uspGetAllEmployees  
AS
```

```
SELECT LastName, FirstName, JobTitle, Department  
FROM HumanResources.vEmployeeDepartment;
```

Especificar parámetros

Para especificar parámetros en un procedimiento almacenado utilizamos la sintaxis:

@parametro tipo

Ejemplo:

```
CREATE PROCEDURE Production.uspGetList @Product varchar(40), @MaxPrice money,  
@ComparePrice money OUTPUT, @ListPrice money OUT
```

La dirección de un parámetro es de entrada, el valor se pasa al cuerpo del procedimiento (IN por omisión), o de salida (OUT o OUTPUT), el procedimiento devuelve un valor al programa de llamada. El valor predeterminado es un parámetro de entrada.

@Product varchar(40) // Parámetro de entrada

@ListPrice money OUT // Parámetro de salida

Un parámetro se considera opcional si tiene un valor predeterminado especificado cuando se declara. No es necesario proporcionar un valor para un parámetro opcional de una llamada a procedimiento.

@SalesPerson nvarchar(50) = NULL

Para especificar un parámetro de salida, se debe indicar la palabra clave OUTPUT (preferible a OUT). El procedimiento devuelve el valor actual del parámetro de salida al programa de llamada cuando se abandona el procedimiento. El programa de llamada también debe usar la palabra clave OUTPUT al ejecutar el procedimiento, a fin de guardar el valor del parámetro en una variable que se pueda usar en el programa de llamada.

La variable OUTPUT debe definirse durante la creación del procedimiento y también durante el uso de la variable. El nombre del parámetro y de la variable no tienen por qué coincidir. Pero el tipo de datos y la posición de los parámetros deben coincidir (a menos que se use @listprice= variable).

Ejemplo:

```
CREATE PROCEDURE Production.uspGetList @Product varchar(40), @MaxPrice money,  
@ComparePrice money OUTPUT, @ListPrice money OUT
```

Valor de retorno

Cada procedimiento almacenado devuelve un código de retorno de tipo entero a quien realiza la llamada. Si el procedimiento almacenado no establece explícitamente un valor para el código de retorno, éste es 0.

Ejemplo:

```
CREATE PROCEDURE SampleProcedure @EmployeeIDParm INT, @MaxTotal INT OUTPUT
...
RETURN @ErrorSave
```

Modificar un procedimiento almacenado

Para modificar un procedimiento almacenado previamente creado:

ALTER PROCEDURE nombre procedimiento ...

Eliminar un procedimiento almacenado

Para borrar un procedimiento almacenado previamente creado:

DROP PROCEDURE procedimiento;

Ejecutar un procedimiento almacenado

Para ejecutar un procedimiento almacenado previamente creado:

EXEC procedure;

EXECUTE procedure;

Ejemplos:

EXEC dbo.uspGetEmployeeManagers @BusinessEntityID = 50;

EXEC AdventureWorks2012.dbo.uspGetEmployeeManagers 50;

EXEC Sales.uspGetSalesYTD N'Blythe';

EXEC dbo.uspGetWhereUsedProductID 819, '20050225'; //Constantes

EXEC dbo.uspGetWhereUsedProductID @ProductID, @CheckDate; //Variables

Atención: si delante de una cadena se indica N (N'string'), la cadena se interpreta como del tipo de datos nvarchar (2 bytes por carácter en lugar de 1 byte por carácter en varchar).

La asignación de los valores para cada uno en una llamada a procedimiento permite proporcionar los parámetros en cualquier orden:

EXECUTE procedimiento @second = 2, @first = 1, @third = 3;

Si un valor de parámetro se proporciona con el formato **@parámetro = valor** todos los parámetros posteriores se deben proporcionar de esta manera. Si los valores de parámetro no se pasan con el formato @parámetro = valor, los valores se deben proporcionar en el orden idéntico (de izquierda a derecha) en el que los parámetros se enumeran en la instrucción CREATE PROCEDURE.

Ejemplos:

```
CREATE PROC ProcWithParameters @name NVARCHAR(50), @color NVARCHAR(15)
AS
SELECT ProductKey, EnglishProductName, Color FROM [dbo].[DimProduct]
WHERE EnglishProductName LIKE @name
AND Color = @color;
```

```
EXEC ProcWithParameters N'%arm%', N'Black';
```

```
EXEC ProcWithParameters @name = N'%arm%', @color = N'Black';
```

```
EXEC ProcWithParameters @color = N'Black', @name = N'%arm%';
```

2. MySQL

Para crear una función almacenada usamos la sintaxis que se describe a continuación:

```
CREATE [OR REPLACE] FUNCTION nombre(parámetros)
RETURNS tipo
BEGIN
    cuerpo;
    RETURN valor;
END
```

Para crear un procedimiento almacenada usamos la sintaxis que se describe a continuación:

```
CREATE [OR REPLACE] PROCEDURE nombre(parámetros)
BEGIN
    cuerpo;
END
```

Ejemplo:

```
CREATE PROCEDURE p (OUT ver_param VARCHAR(25), INOUT incr_param INT)
BEGIN
    # Set value of OUT parameter
    SELECT VERSION() INTO ver_param;
    # Increment value of INOUT parameter
    SET incr_param = incr_param + 1;
END;
```

Especificar parámetros

Especificar un parámetro como **IN**, **OUT** o **INOUT** solo es válido para un PROCEDURE. Para una FUNCIÓN, los parámetros siempre se consideran parámetros IN.

[**IN** | **OUT** | **INOUT**] param_name type

Ejemplo:

```
CREATE PROCEDURE p (OUT ver_param VARCHAR(25), INOUT incr_param INT)
BEGIN
  # Set value of OUT parameter
  SELECT VERSION() INTO ver_param;
  # Increment value of INOUT parameter
  SET incr_param = incr_param + 1;
END;
```

Valor de retorno

La cláusula RETURNS se puede especificar solo para una FUNCTION, siendo obligatoria. Indica el tipo de retorno de la función y el cuerpo de la función debe contener una declaración de valor RETURN. Si la instrucción RETURN devuelve un valor de un tipo diferente, el valor se convierte en el tipo adecuado.

Ejemplo:

```
CREATE FUNCTION hello (s CHAR(20))
RETURNS CHAR(50)
RETURN CONCAT('Hello, ',s,'!');

SELECT hello('world'); // Hello, World!
```

Modificar un procedimiento almacenado

Para modificar una función almacenada previamente creado:

ALTER FUNCTION nombrefuncion ...

Para modificar un procedimiento almacenado previamente creado:

ALTER PROCEDURE nombreprocedimiento ...

Eliminar un procedimiento almacenado

Para borrar un procedimiento almacenado previamente creado:

DROP {PROCEDURE | FUNCTION} [IF EXISTS] sp_name

Ejecutar un procedimiento almacenado

Para ejecutar un procedimiento almacenado previamente creado:

CALL sp_name([parameter[,...]])

CALL sp_name[()]

Para cada parámetro OUT o INOUT, pasamos una variable definida por el usuario en la instrucción CALL que invoca el procedimiento para que podamos obtener su valor cuando el procedimiento regrese.

Ejemplo:

```
CREATE PROCEDURE citycount (IN country CHAR(3), OUT cities INT)
BEGIN
    SELECT COUNT(*) INTO cities FROM world.city
    WHERE CountryCode = country;
END
```

CALL citycount('JPN', @cities); -- ciudades en Japón

CALL citycount('FRA', @cities); -- ciudades en Francia

3. Oracle

Oracle utiliza PL/SQL. Para crear una función almacenada usamos la sintaxis que se describe a continuación:

```
CREATE [OR REPLACE] FUNCTION name_func (parameter_name, ...)
RETURN tipo_dato
IS
    -- Declaración variables locales
BEGIN
    -- Sentencias
    RETURN valor;
END;
```

Para crear un procedimiento almacenada usamos la sintaxis que se describe a continuación:

```
CREATE [OR REPLACE] PROCEDURE name_proc (parameter_name, ...)
IS
    -- Declaración variables locales
BEGIN
    -- Sentencias
END;
```

Ejemplo:

```
CREATE FUNCTION sal_ok (salary REAL, title REAL) RETURN BOOLEAN IS
    min_sal REAL;
    max_sal REAL;
BEGIN
    SELECT losal, hisal INTO min_sal, max_sal
    FROM sals
    WHERE job = title;
    RETURN (salary >= min_sal) AND (salary <= max_sal);
END;
```

Especificar parámetros

Especificar un parámetro como **IN**, **OUT** o **IN OUT**. Cada parámetro tiene la siguiente sintaxis:

parameter_name [IN | OUT | IN OUT] datatype_name

Ejemplo:

```
CREATE OR REPLACE PROCEDURE grabar_cliente(id IN OUT INT, nombre IN VARCHAR2,  
apellidos IN VARCHAR2, fecha IN DATE)  
IS  
    contador INT;  
BEGIN  
    ...  
END;
```

Modificar un procedimiento almacenado

Para modificar una función almacenada previamente creado:

ALTER FUNCTION nombrefuncion ...

Para modificar un procedimiento almacenado previamente creado:

ALTER PROCEDURE nombreprocedimiento ...

Eliminar un procedimiento almacenado

Para borrar un procedimiento almacenado previamente creado:

DROP {PROCEDURE | FUNCTION} sp_name

Ejecutar un procedimiento almacenado

Para ejecutar un procedimiento almacenado previamente creado:

EXECUTE sp_name([parameter[,...]])

EXEC sp_name[()]

Ejemplo:

EXECUTE incrementar(30);

EXEC insertar('Tres',20);

3. EVENTOS Y DISPARADORES

Un disparador o **trigger** es un procedimiento almacenado que se ejecuta automáticamente cuando se cumple una condición preestablecida al realizar una operación. En otras palabras, son reglas que especifican acciones que son activadas automáticamente por determinados eventos.

Dependiendo de la base de datos, los disparadores pueden ser ejecutados cuando suceden ciertos eventos concretos: una inserción (INSERT), una actualización (UPDATE) o un borrado (DELETE).

Así pues, los triggers se ejecutan automáticamente como respuesta a un evento concreto, sin necesidad de interacción por parte del usuario.

Su utilidad principal es implementar reglas de negocio (lógica del negocio). Por ejemplo, se pueden utilizar para asignar valores a campos calculados, para generar históricos, para establecer restricciones en los datos de una tabla o para preservar la semántica del negocio (relaciones exclusivas).

Sintaxis básica:

```
CREATE [OR REPLACE] TRIGGER nombre  
[temporalidad del evento]  
[granularidad del evento]  
[WHEN condición]  
BEGIN  
cuerpo del trigger  
END;
```

Existen dos tipos de triggers (granularidad):

- **A nivel de fila (FOR EACH ROW):** se ejecuta por cada una de las filas de la tabla afectadas.
- **A nivel de sentencia (FOR EACH STATEMENT):** se ejecuta una única vez, por lo que afecta a toda la tabla.

La estructura de un trigger es:

- **Evento:** definición del trigger.

```
CREATE [OR REPLACE] TRIGGER nombre  
[temporalidad del evento]  
[granularidad del evento]
```

La temporalidad hace referencia a si el disparador se ejecuta antes (**BEFORE**) o después (**AFTER**) del evento que causa la activación del trigger. Puede ocurrir que el disparador

ejecute su acción en sustitución de la sentencia correspondiente (**INSTEAD OF**), muy útil para las vistas (borrado de datos).

La granularidad hace referencia a los dos tipos de triggers vistos: a nivel de fila o de sentencia. En los triggers a nivel de fila se pueden indicar campos concretos de una tabla (solo para UPDATE) de tal forma que solo se ejecutará el trigger si son modificados algunos o todos los campos de esa lista.

Ejemplo: Se ejecuta el trigger al modificarse alguno o todos los campos de la lista.

AFTER UPDATE OF A, B, C ON TABLE → Si se modifica A o B o C de TABLE.

BEFORE UPDATE D, E ON TABLE → Si se modifica D o E de TABLE

- **Condición**: condición a satisfacer para que se dispare.

[**WHEN** condición]

La condición solo se aplica cuando el disparador es a nivel de fila. Usamos **:OLD** para hacer referencia el valor antiguo de una fila y **:NEW** para el nuevo valor. Resaltar que ante un INSERT **:OLD** no tendrá sentido, al igual que ante un DELETE **:NEW** carecerá de valor.

Ejemplo: Se ejecuta el trigger al modificarse alguno o todos los campos de la lista.

WHEN :new.salario > 1500

- **Acción**: tareas que realiza el trigger.

BEGIN

cuerpo del trigger

END;

Ejemplo: trigger a nivel de fila que se ejecuta después de actualizarse uno de los campos (A o B) o ambos.

```
CREATE OR REPLACE TRIGGER actualizar
AFTER UPDATE OF A, B ON tabla
FOR EACH ROW
BEGIN
    INSERT INTO log (A, B) VALUES(:new.A, :new.B);
END actualizar;
```

4. ESTÁNDARES DE CONECTIVIDAD: ODBC Y JDBC

ODBC (Open Database Connectivity) es una API estándar de Microsoft de acceso a base de datos que permite a las aplicaciones acceder a los datos independientemente del origen de datos o SGBD concreto (tanto relacionales como no relacionales).

El objetivo de ODBC es hacer posible el acceder a cualquier dato desde cualquier aplicación, sin importar qué SGBD concreto almacene los datos.

Se basa en las especificaciones de Call Level Interface (CLI) de Open Group e ISO/IEC 9075-3:1995 para las API de base de datos y usa SQL como lenguaje de acceso a bases de datos.

ODBC está diseñado para lograr la máxima interoperabilidad, es decir, la capacidad de una aplicación para acceder a diferentes SGBD con el mismo código fuente.

Las funciones de la API ODBC son implementadas los desarrolladores de controladores específicos de cada SGBD. Las aplicaciones llaman a las funciones de estos controladores para acceder a los datos de forma independiente de SGBD.

Para cada SGBD hay un **driver ODBC** específico.

JDBC (Java Database Connectivity) es una API de Java que proporciona acceso universal a datos desde el lenguaje de programación Java.

Con la API de JDBC, que proporciona una interfaz común, se puede acceder a prácticamente cualquier fuente de datos, desde bases de datos relacionales hasta hojas de cálculo y archivos sin formato. La API de JDBC se compone de dos paquetes:

- java.sql.
- javax.sql.

Ejemplo: uso de la misma interfaz para distintos tipos de bases de datos.

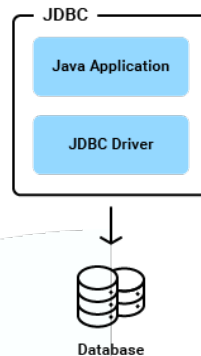
```
Connection conexion;
String driver = "com.mysql.jdbc.Driver";
String url = "jdbc:mysql://localhost:3306/";
String nombreBD = "universidad";
String user = "pruebas";
String pass = "pruebas";

// CONEXIÓN CON SGBD
conexion = DriverManager.getConnection(url + nombreBD, user, pass);

// PREPARACIÓN CONSULTA
Statement stmt = conexion.createStatement();
String select = "SELECT * FROM asignaturas;";

// EJECUCIÓN CONSULTA
ResultSet rs = stmt.executeQuery(select);
```

Para utilizar la API de JDBC con un SGBD concreto, se necesita un **driver** basado en tecnología JDBC como intermediario entre la tecnología JDBC y la base de datos.



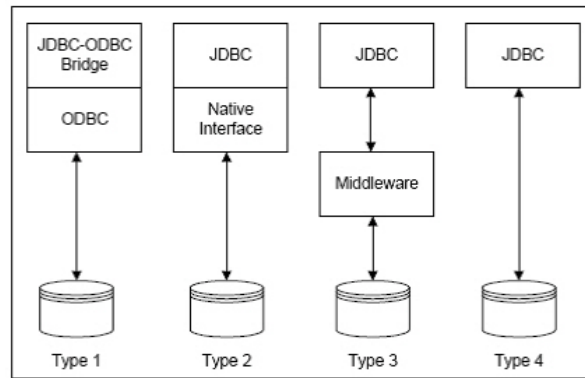
Conviene resaltar que el paquete [java.sql](#) (API de JDBC) está formado principalmente por un conjunto de interfaces y que serán los drivers JDBC los encargados de implementar esos métodos.

Interface Summary	
Interface	Description
Array	The mapping in the Java programming language for the SQL type ARRAY.
Blob	The representation (mapping) in the Java™ programming language of an SQL BLOB value.
CallableStatement	The interface used to execute SQL stored procedures.
Clob	The mapping in the Java™ programming language for the SQL CLOB type.
Connection	A connection (session) with a specific database.
DatabaseMetaData	Comprehensive information about the database as a whole.
Driver	The interface that every driver class must implement.
NClob	The mapping in the Java™ programming language for the SQL NCLOB type.
ParameterMetaData	An object that can be used to get information about the types and properties for each parameter marker in a PreparedStatement object.
PreparedStatement	An object that represents a precompiled SQL statement.
Ref	The mapping in the Java programming language of an SQL REF value, which is a reference to an SQL structured type value in the database.
ResultSet	A table of data representing a database result set, which is usually generated by executing a statement that queries the database.
ResultSetMetaData	An object that can be used to get information about the types and properties of the columns in a ResultSet object.
Rowid	The representation (mapping) in the Java programming language of an SQL ROWID value.
Savepoint	The representation of a savepoint, which is a point within the current transaction that can be referenced from the Connection.rollback method.
SQLData	The interface used for the custom mapping of an SQL user-defined type (UDT) to a class in the Java programming language.
SQLInput	An input stream that contains a stream of values representing an instance of an SQL structured type or an SQL distinct type.
SQLOutput	The output stream for writing the attributes of a user-defined type back to the database.
SQLXML	The mapping in the Java™ programming language for the SQL XML type.
Statement	The object used for executing a static SQL statement and returning the results it produces.
Struct	The standard mapping in the Java programming language for an SQL structured type.
Wrapper	Interface for JDBC classes which provide the ability to retrieve the delegate instance when the instance in question is in fact a proxy class.

Así pues, para establecer una conexión con un SGBD específico, como Oracle o MySQL, será el driver específico el que implemente las interfaces para la conexión con el SGBD y para interactuar con él. Por tanto, se requiere la instalación del driver concreto en la máquina que se ejecuta la app de Java.

Existen 4 tipos de drivers JDBC:

- **Tipo 1 Puente JDBC-ODBC**: el driver convierte las llamadas a métodos a la API de ODBC.
- **Tipo 2 API nativa**: el driver convierte las llamadas a métodos de la API específica de la base de datos.
- **Tipo 3 Middleware**: se utiliza un servidor intermedio entre el driver y la base de datos.
- **Tipo 4 Java puro**: el driver convierte las llamadas directamente al protocolo específico de la base de datos.

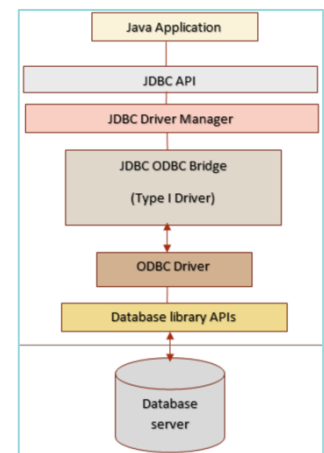


Driver Tipo 1 Bridge JDBC-ODBC

La API de Java traduce las llamadas a la tecnología ODBC. Se requiere tener instalado el driver ODBC en la máquina cliente.

Ventajas: acceso a casi todos los SGBD ya que los drivers ODBC están muy extendidos.

Desventajas: ineficiente al existir varios niveles de software, no recomendado si el número de transacciones es elevado.

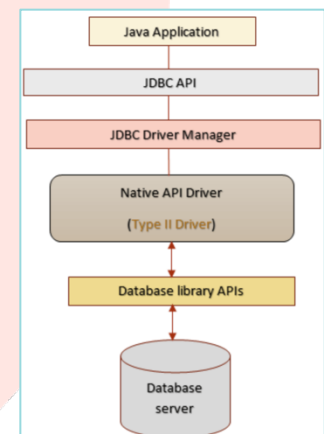


Driver Tipo 2 API nativa

No se requiere de un driver ODBC. Las llamadas JDBC se traducen en llamadas específicas del API de la base de datos. Cada cliente debe tener instalado el driver.

Ventajas: más eficiente que el driver Tipo 1.

Desventajas: ligado al SGBD elegido.

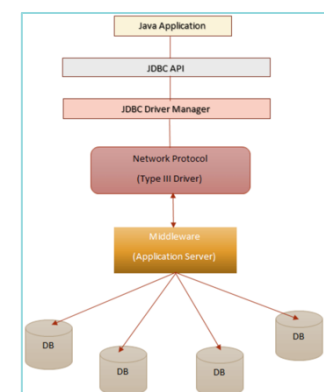


Driver Tipo 3 Middleware

El driver JDBC convierte las llamadas a un protocolo independiente de la base de datos y las envía al servidor intermedio (middleware). Este servidor, que es una capa de abstracción, traslada las peticiones al SGBD concreto.

Ventajas: el driver no depende del SGBD específico, con un solo driver se puede manejar cualquier base de datos (siempre que el middleware lo admita).

Desventajas: el middleware añade un nivel más de software disminuyendo el rendimiento.



Driver tipo 4 Java puro

El driver nativo, desarrollado 100% en Java, es específico para cada SGBD. Así pues, el fabricante del SGBD proporcionará su driver JDBC de tipo 4.

La llamada JDBC se traduce directamente en una a la base de datos, sin intermediarios.

Ventajas: muy eficiente al no existir varios niveles de software.

Desventajas: ligado al SGBD elegido.

